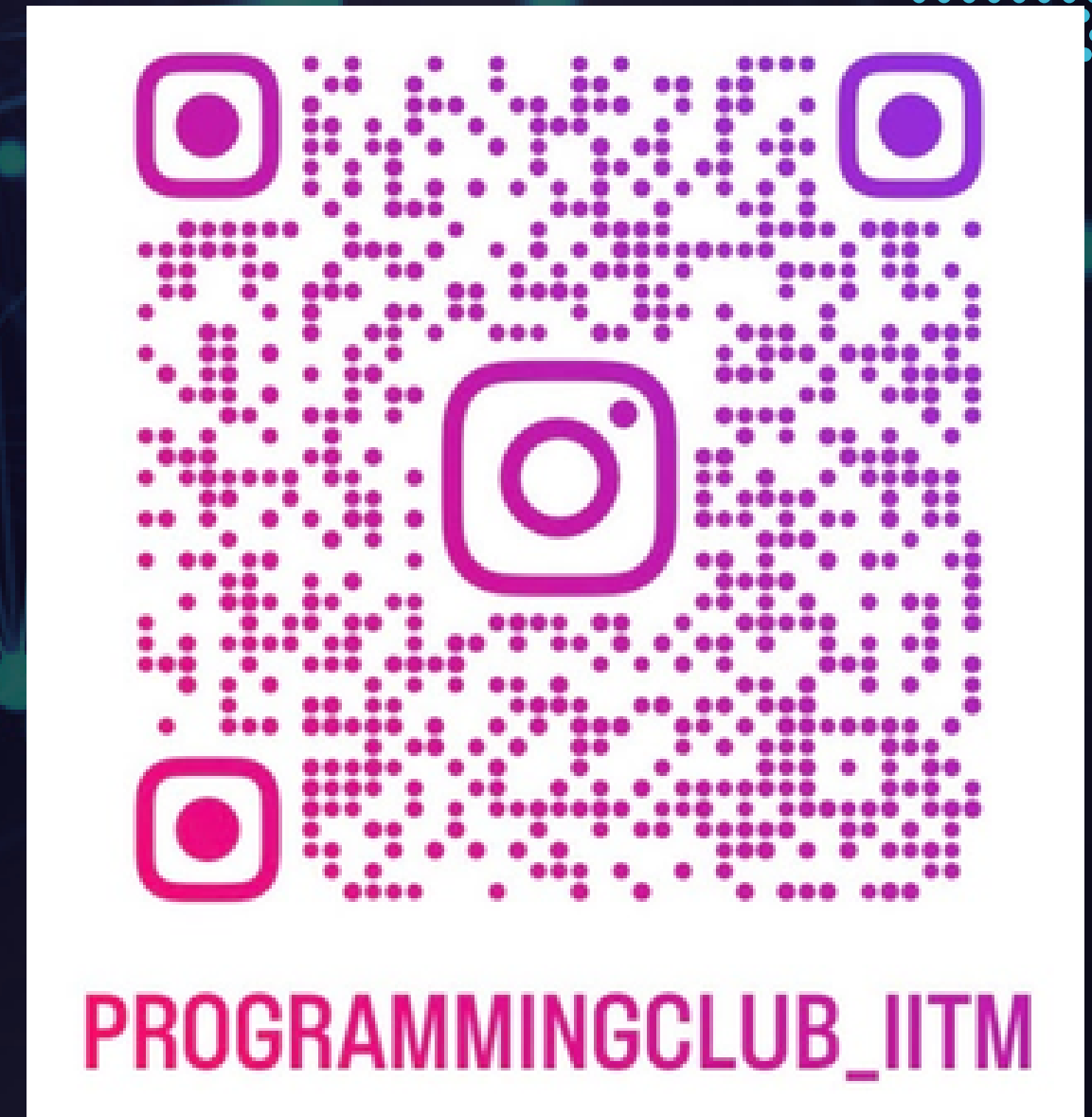




PROGRAMMING CLUB



<https://progclubiitm.com/>



PROGRAMMING CLUB

presents:

INTRODUCTION TO
PROGRAMMING

CPS-1

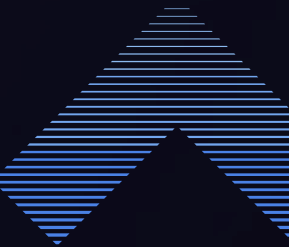


CP

&

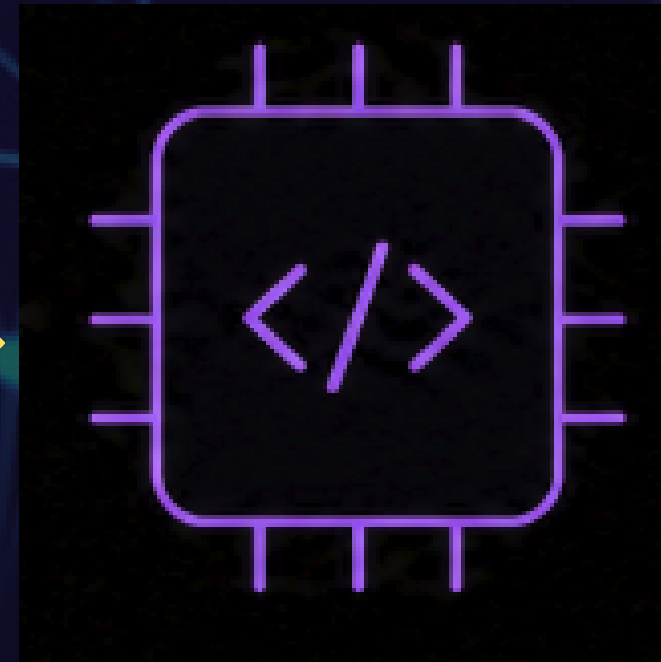


CPP



What is a PROGRAMMING LANGUAGE

```
int x = 10  
x = x + 5  
print(x);
```



WRITE INSTRUCTIONS
IN A PROGRAMMING
LANGUAGE

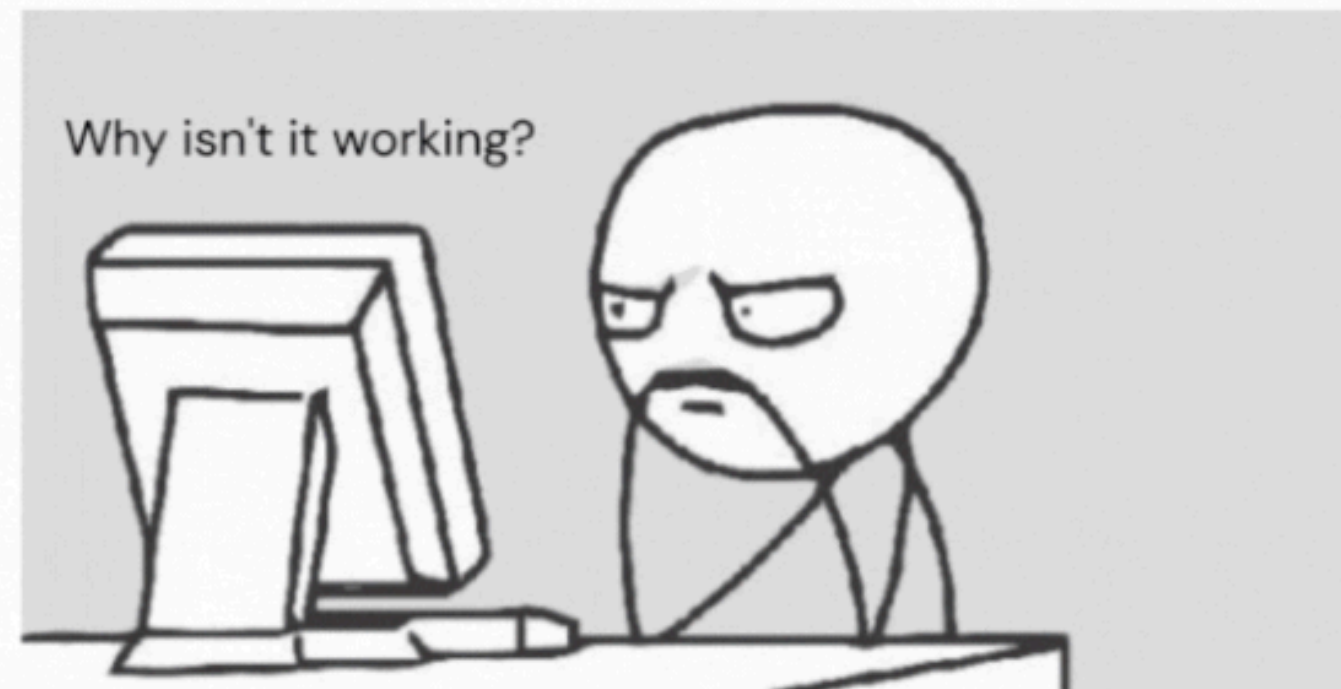
COMPUTER
UNDERSTANDS IT
AND TRANSLATES IT

GIVES REQUIRED
OUTPUT

WHAT PEOPLE THINK PROGRAMMING IS LIKE



WHAT IT IS REALLY LIKE



WHY C++

- **It's fast:** C++ programs run very quickly compared to many other languages
- **It shows you what's under the hood:**
You have to manage memory yourself - gives you more control (and builds character)
- **It has many real-world applications:**
Web browsers, videogames, quant trading systems etc.
- **It's good for competitive programming:**
Contests have strict time limits, and C++ is both fast and comes with a toolbox of ready-made tools (STL)



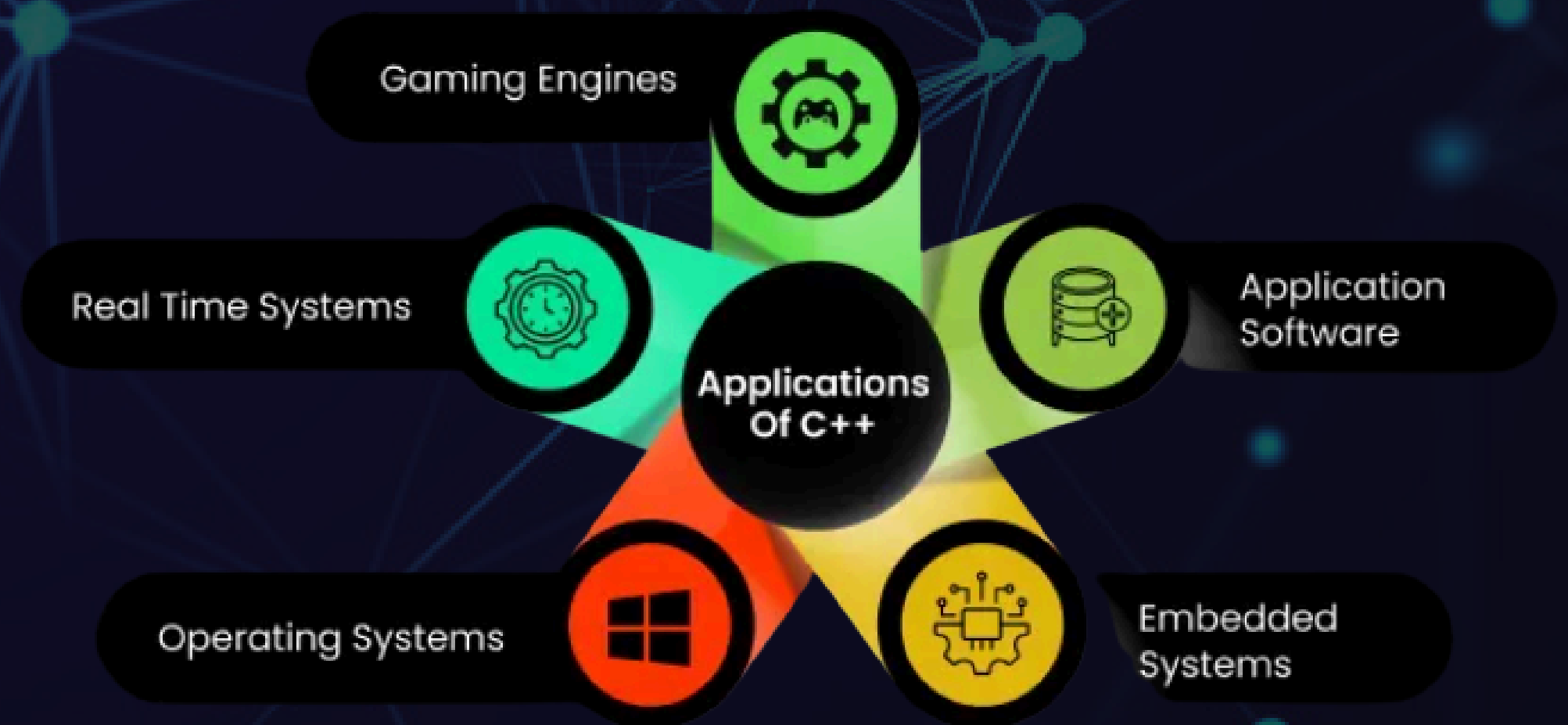
WELCOME TO C++

THE BASICS

**SYNTAX: – SET OF RULES THAT DICTATE HOW A
PIECE OF CODE MUST BE WRITTEN**

```
#include<iostream>

int main(){
    return 0;
}
```



WELCOME TO C++

THE BASICS

```
#include <bits/stdc++.h>
#define pi 3.14
using namespace std;

int main(){
    return 0;
}
```

PREPROCESSOR DIRECTIVES

FILE
INCLUDE

#include

MACROS

#define

CONDITIONAL
&
OTHERS



WELCOME TO C++

THE BASICS

MAIN FUNCTION

```
#include <bits/stdc++.h>
using namespace std;

int main(){

    return 0;

}
```

- First to execute in the program
- int main is convention
- int function has to return integer, so 0 is convention.
- After returning it will exit the function

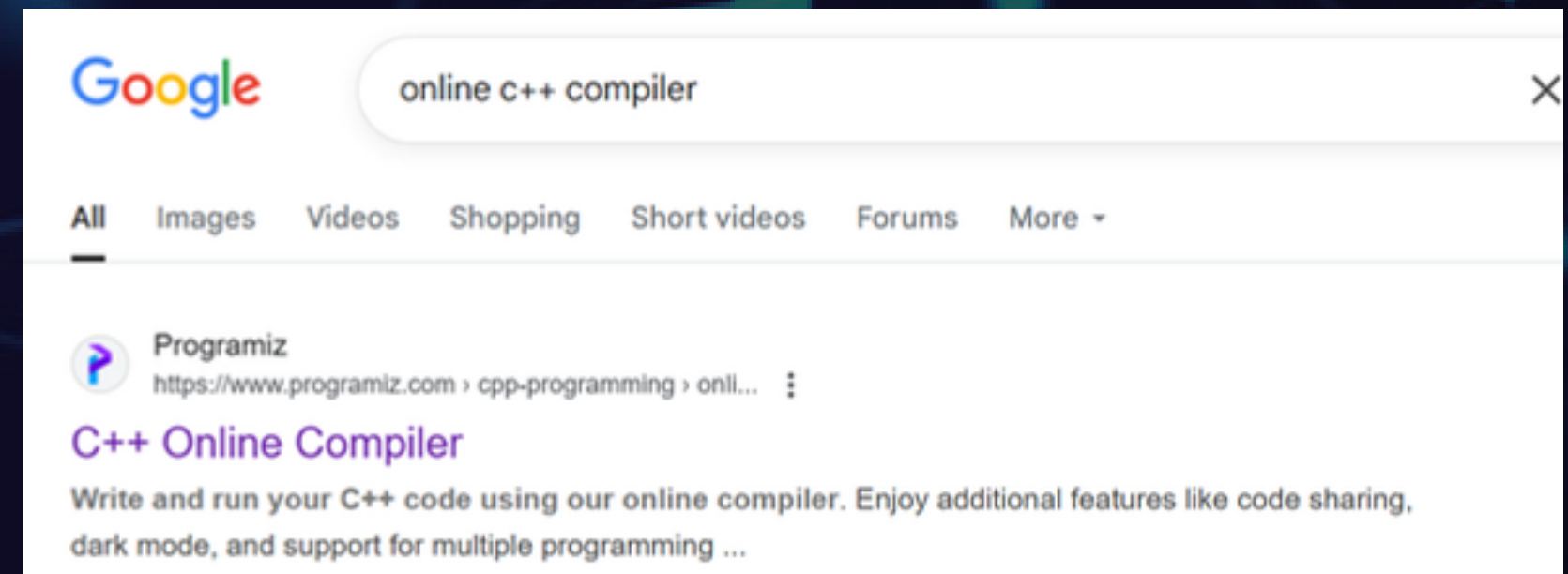
WELCOME TO C++

THE BASICS

```
#include <bits/stdc++.h>
using namespace std;

int main(){

    return 0;
}
```



**APPLICATIONS
LIKE VSCODE**

VARIABLES

What is a Variable?

- A named memory location used to store data values that your program can manipulate during execution

Basic Syntax in C++:

```
type variableName = value;
```

Ex:

```
int num = 18;  
string s = "Virat";
```

**A variable is like
a box with a label**



DECLARATION VS INITIALIZATION

Declaration - tells the compiler that a variable exists and what type it is

- Memory is reserved but no value is set yet
- The value is garbage [whatever junk was already in that memory]

```
int x;
```

```
int a, b, c;
```

Initialization - giving the variable its first actual value

- Memory is reserved and a value is assigned in one step

```
int x = 5;
```

```
int a = 1, b = 2;
```

VARIABLE NAMING RULES

- Must start with a letter or underscore _ (not a digit)
- No whitespace or special characters (#, \$, %, *, etc.)
- Case-sensitive →
 - variable_name ≠ Variable_name
- Cannot use C++ keywords (float, class, return, int, etc.) as a name

```
int variable_name = 5;
```

✓ Valid

```
int Variable_name = 5;
```

✓ Different var (case-sensitive)

```
int variable name = 5;
```

x No space allowed

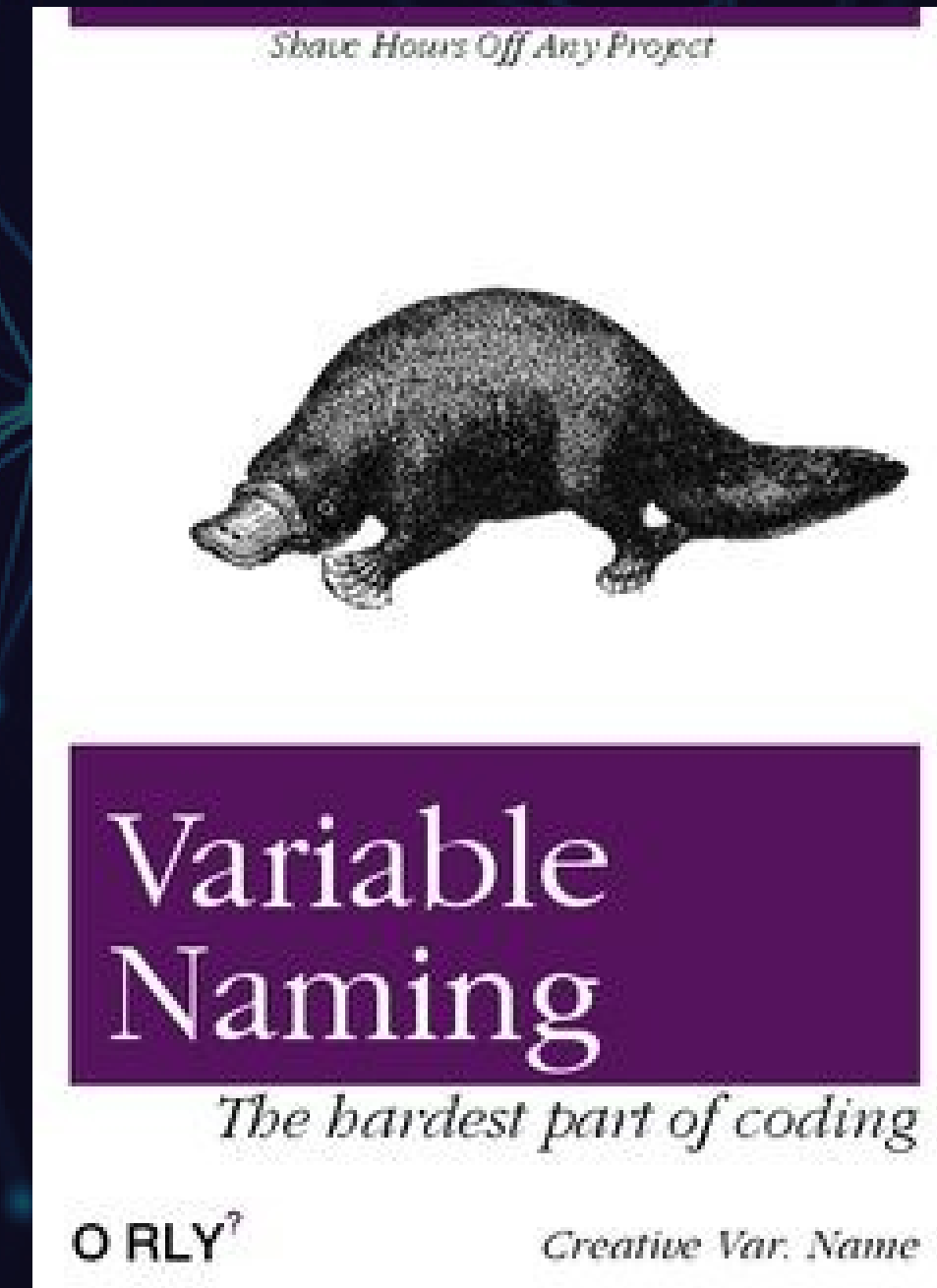
```
int class = 5;
```

x Reserved keyword

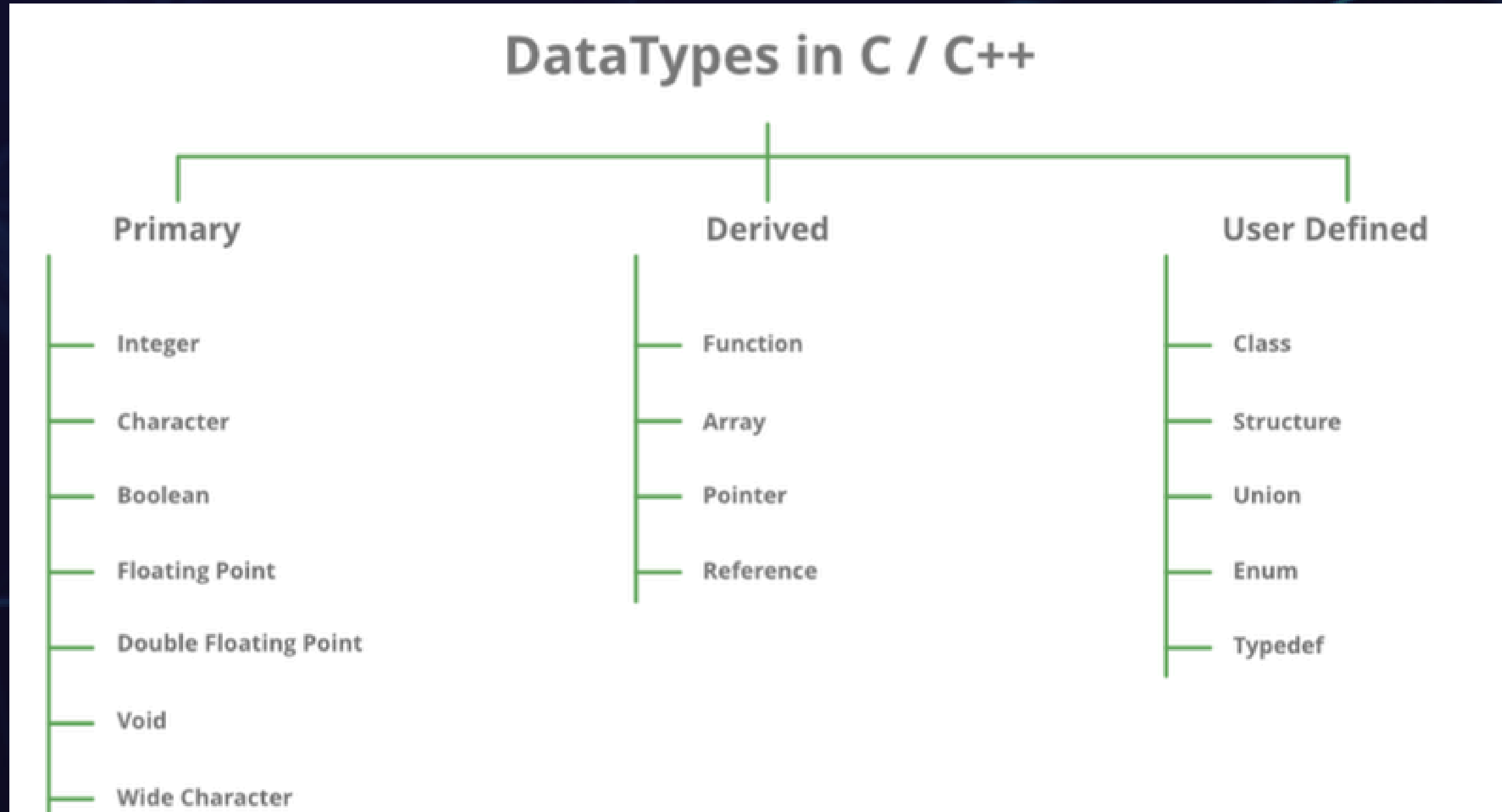
VARIABLE NAMING CONVENTION



- Use meaningful, descriptive names → `numStudents` instead of `x1`
- Common styles: snake_case [`num_students`] or camelCase [`numStudents`] – pick one and stay consistent
- Keep names reasonably short but clear – not `n`, but also not `theTotalNumberOfStudentsEnrolledInClass`



DATA TYPES



DATA TYPES

INT

Data Type	Size (In Bytes)	Range
short int	2	-32,768 to 32,767
unsigned short int	2	0 to 65,535
unsigned int	4	0 to 4,294,967,295
int	4	-2,147,483,648 to 2,147,483,647
long int	4	-2,147,483,648 to 2,147,483,647
unsigned long int	4	0 to 4,294,967,295
long long int	8	$-(2^{63})$ to $(2^{63})-1$
unsigned long long int	8	0 to 18,446,744,073,709,551,615
signed char	1	-128 to 127
unsigned char	1	0 to 255

DOUBLE

float	4
double	8
long double	12

**CAN STORE DECIMAL
VALUES LIKE 5.2, 3.14**

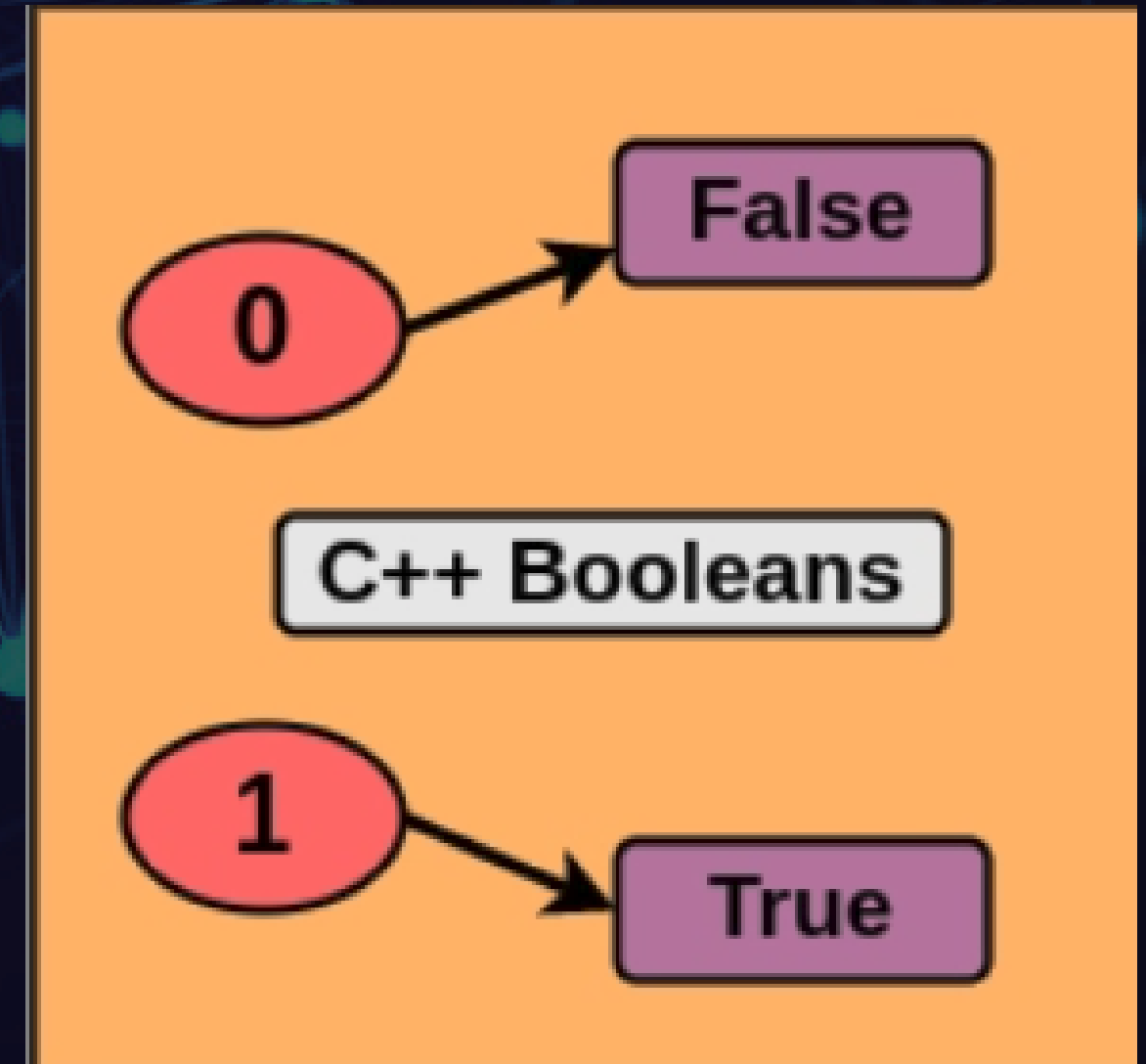
DATA TYPES

STRING

```
#include <bits/stdc++.h>
using namespace std;

int main(){
    string str = "C++";
    return 0;
}
```

BOOLEAN

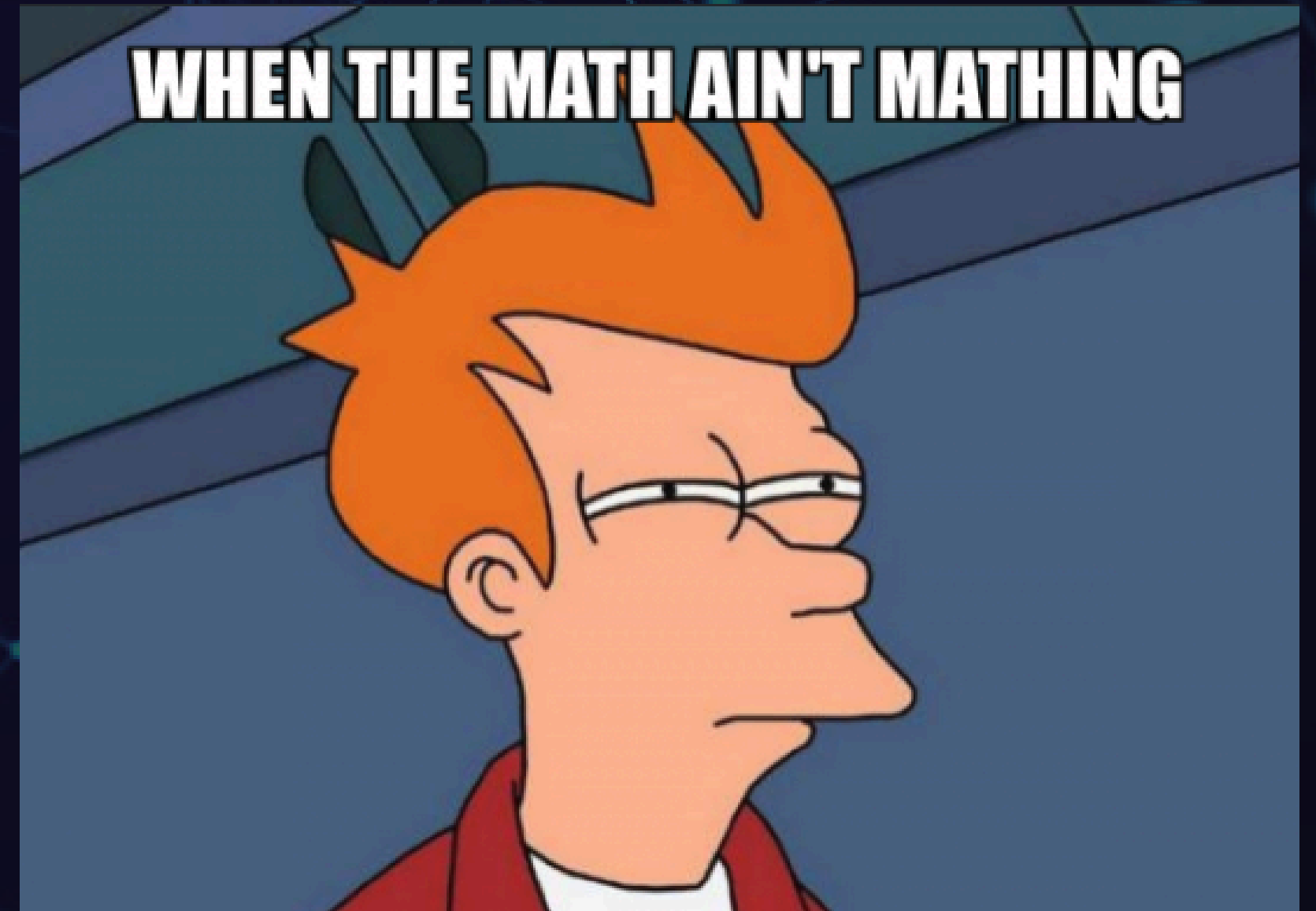


```
#include <bits/stdc++.h>
using namespace std;

int main(){
    double a = 0.1;
    double b = 0.2;
    double c = 0.3;
    bool ans = (a + b) == c;
    //Here == is comparing if (a + b)
    //is equal to c
    cout << ans << endl;
    //cout is for printing
    return 0;
}
```

```
g++ sample.cpp
./a.out
0
```

WHEN THE MATH AIN'T MATHING



OPERATORS

```
int x = 5;
```

```
int y = x + 3;
```

```
int z = y = x + 2;
```

What do you
think is the
value of z?

OPERATORS

Arithmetic Operators

+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Remainder

Assignment Operators

=	Simple assignment
++	Increment
--	Decrement
+=, -=, *=, /=, %=	Combined assignment operators (e.g., x += 5 is equivalent to x = x + 5)

Relational Operators

==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

Bitwise Operators

&	Bitwise AND
	Bitwise OR
^	Bitwise XOR
~	Bitwise NOT
<<	Left shift
>>	Right shift

Logical Operators

&&	Logical AND
	Logical OR
!	Logical NOT

OPERATORS

```
int a = b + 3;
```

```
a += 5;
```

SYNTAX

<variable1>
/constant

[operator]

<variable2>
/constant



operation(variable1, variable2)

Some change **can** happen to variable1 and variable2.

And they **can** give out a value, which can be used by other operators

Binary operators are operators that require two operands

SYNTAX

OPERATORS

<variable1> [operator]

operation(variable1)

Post-Increment

```
int x = 5;  
int y = x++;
```

Pre-Increment

```
int x = 5;  
int y = ++x;
```

Some change **can** happen to variable1.
And they **can** give out a value, which can
be used by other operators

*Unary operators are operators that only
require one operand*

OPERATORS

```
int z = y = x + 2;
```

Most operators are from
LEFT to RIGHT
But assign(=) and unary
operators are
RIGHT to LEFT

When unsure of order, use parentheses `()` to make it explicit,
for both the compiler and anyone reading your code later.



INPUT/OUTPUT



```
cin >> x;
```

```
cout << x;
```

```
cout << "Hello World!";
```



INPUT/OUTPUT

Extraction operator

Object
of type
istream

```
cin >> x;
```

```
cout << x;
```

```
cout << "Hello World!";
```

Object
of type
ostream

Insertion operator

Just keep
in the
back of
your mind

INPUT/OUTPUT

Multiple inputs and outputs with one cin/cout

```
#include <bits/stdc++.h>
using namespace std;

int main(){
    int a, b, sum;
    cout << "Enter two numbers a and b:" << endl;

    cin >> a >> b;
    sum = a + b;
    cout << "Sum of the two numbers: " << sum << endl;

    return 0;
}
```

```
Enter two numbers a and b:
5 3
Sum of the two numbers: 8
```

SWAP NUMBERS

Using a temporary variable

```
int a = 5;  
int b = 10;  
  
int temp = a;  
a = b;  
b = temp;
```

Now, try doing it without using an extra variable.

```
int a = 5;  
int b = 10;  
  
// do some magic here
```

SWAP NUMBERS

Using a temporary variable

```
int a = 5;  
int b = 10;  
  
int temp = a;  
a = b;  
b = temp;
```

Now, try doing it without using an extra variable.

```
int a = 5;  
int b = 10;  
  
// do some magic here  
a = a + b;  
b = a - b;  
a = a - b;
```

CONDITIONALS

if else

```
if(condition1){  
    /*Code for condition 1*/  
}  
else if(condition2){  
    /*Code for condition 2*/  
}  
else if(condition3){  
    /*Code for condition 3*/  
}  
else{  
    /*Code if no condition is met*/  
}
```

Nested if else

```
if(condition1){  
    if(condition1_1){  
        /*Code 1_1*/  
    }  
    else if(condition1_2){  
        /*Code 1_2*/  
    }  
    else{  
        /*Code 1_3*/  
    }  
}  
else{  
    /*Code 2*/  
}
```

CONDITIONALS

Write code to input two positive integers a, b and find if their sum is odd or even and print both sum and parity

**For example:
if $a = 5$, $b = 3$
You need to print:
8 Even**

CONDITIONALS

Write code to input two integers a, b and find if their sum is odd or even and print both sum and parity

```
#include <bits/stdc++.h>
using namespace std;

int main(){
    int a, b;
    cin >> a >> b;
    int sum = a + b;
    if(sum % 2 == 1){
        cout << sum << " Odd" << endl;
    }
    else{
        cout << sum << " Even" << endl;
    }
}
```

Your code probably looks something like this

CONDITIONALS

Write code to input two integers a, b and find if their sum is odd or even and print both sum and parity

**Single line if-else code
Ternary operator**

```
#include <bits/stdc++.h>
using namespace std;

int main(){
    int a, b;
    cin >> a >> b;
    int sum = a + b;
    if(sum % 2 == 1){
        cout << sum << " Odd" << endl;
    }
    else{
        cout << sum << " Even" << endl;
    }
}
```

```
((condition) ? (if True) : (if False));
((sum % 2) == 1) ? cout << "Odd" : cout << "Even";
```

-5 -2
-7 Even

```
#include <bits/stdc++.h>
using namespace std;

int main(){
    int a, b;
    cin >> a >> b;
    int sum = a + b;
    if(sum % 2 == 1){
        cout << sum << " Odd" << endl;
    }
    else{
        cout << sum << " Even" << endl;
    }
}
```

**When you know your
logic is right, but the
program says otherwise**



LOOPS

For loop

```
for (initialization; condition; increment/decrement)
    // code to be executed
}
```

Number of iterations
known

While loop

```
while (condition) {
    // code to be executed
}
```

Number of iterations
may not be known

There are also do-while
loops -
try it yourself (HW lol)

LOOPS

Example: calculate factorial of a number

```
#include <iostream>
using namespace std;

int main(){
    int n; cin >> n;
    int output = 1;
    for(int i = 1; i <= n; i++) {
        output *= i;
    }
    cout << n << "! is " << output << endl;

    return 0;
}
```

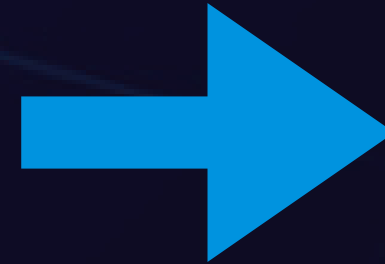
NESTED LOOPS

loop inside loop

```
#include <iostream>
using namespace std;

int main(){
    for (int i = 0; i < 5; i++) {
        for (int j = 0; j < 3; j++) {
            cout<<'('<<i<<', '<<j<<" ) ";
        }
        cout<<endl;
    }

    return 0;
}
```



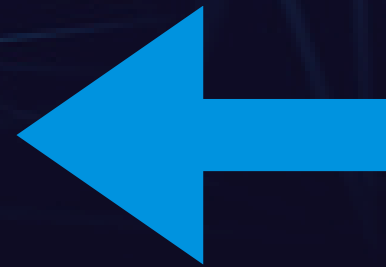
(0,0)	(0,1)	(0,2)
(1,0)	(1,1)	(1,2)
(2,0)	(2,1)	(2,2)
(3,0)	(3,1)	(3,2)
(4,0)	(4,1)	(4,2)

NESTED LOOPS

Break

Terminates the
smallest enclosing loop

```
Trying 1
Trying 2
Trying 3
Trying 4
Trying 5
Trying 6
Trying 7
Correct guess!
```



Example: Guess the number

- Loop tries a fixed sequence of guesses
- The moment the guess matches the target, print success and break (stop guessing further)

```
#include <iostream>
using namespace std;

int main(){
    int target = 7;

    for (int guess = 1; guess <= 10; guess++) {
        cout << "Trying " << guess << endl;
        if (guess == target) {
            cout << "Correct guess!" << endl;
            break;
        }
    }

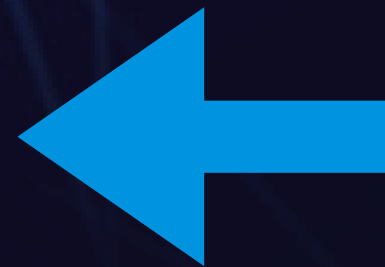
    return 0;
}
```

NESTED LOOPS

Continue

Skips the rest of the loop statement and starts the next iteration

```
0
1 is odd
2
3 is odd
4
```



```
#include <iostream>
using namespace std;

int main(){
    for (int i = 0; i < 5; i++) {
        cout<<endl<<i;
        if ( i % 2 == 0 ) {
            continue;
        }
        cout<<" is odd";
    }cout<<endl;

    return 0;
}
```

SCOPE

```
#include <bits/stdc++.h>
using namespace std;
```

```
int main(){
    {
        int i = 0;
    }
    cout << i << endl;
    return 0;
}
```

```
sample.cpp: In function 'int main()':
sample.cpp:8:13: error: 'i' was not declared in this scope
      8 |         cout << i << endl;
          |                ^
```

Block refers to a sequence of program statements enclosed within curly braces "{ }".

Once the block is exited, variables declared inside are erased from memory

What happens in a block,
stays in a block
-Sung Jin-woo

SCOPE

```
#include <bits/stdc++.h>
using namespace std;
```

```
int main(){
    int i = 5;
    {
        i = 0;
    }
    cout << i << endl;
    return 0;
}
```

Gives '0' as output

Can be accessed and used by inner blocks

'i' can be used anywhere within the int main scope

```
#include <bits/stdc++.h>
using namespace std;
```

```
int main(){
    int i = 5;
    {
        cout << "Before " << i << endl;
        i = 0;
        cout << "After " << i << endl;
    }
    cout << i << endl;
    return 0;
}
```

SCOPE

```
#include <bits/stdc++.h>
using namespace std;
```

```
int main(){
    int x = 5;
    int x = 3;
    return 0;
}
```

It tries to re
declare the same
variable, but gets
confused, as there
is already an x.



```
#include <bits/stdc++.h>
using namespace std;
```

```
int x = 5;
int main(){
    int x = 3;
    return 0;
}
```

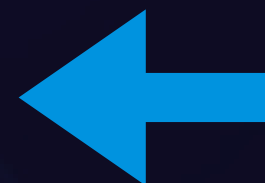
Both are different
variables with same
name, because it is
declared in
different scopes



SCOPE

The compiler considers
the one in the innermost
block(scope) which is
main here

Output: 3



```
#include <bits/stdc++.h>
using namespace std;

int x = 5;
int main(){

    int x = 3;
    cout << x << endl;
    return 0;
}
```

HomeWork

Think of how we can access
the global x now.



CODEFORCES



<https://codeforces.com/>

**THIS IS WHERE WE DO
MOST OF OUR COMPETITIVE
PROGRAMMING (CP)**

CODEFORCES





Sponsored by TON

HOME TOP CATALOG CONTESTS GYM PROBLEMSET GROUPS RATING EDU API CALENDAR HELP

ICPC Challenge powered by Huawei

By ICPCNews, 3 weeks ago, 

Hello, Codeforces!

We are happy to invite you to an exciting online event: ICPC Online Challenge powered by Huawei.

This Challenge features the problem

Scheduling Algorithm for Collaborative Inference System With the rapid advancement of artificial intelligence, large language model (LLM) services are becoming increasingly integrated into people's daily lives and work. At the same time, the economic efficiency and data security of these services are receiving greater attention.

Edge-Cloud Collaborative Inference has emerged as a promising approach to addressing these challenges. In this architecture, LLM inference is divided between a user's on-premises edge environment and the cloud. Using a U-shaped split, the first and last layers of the model run on the edge, while the computationally intensive middle layers run in the cloud. This allows private inputs and outputs to remain on the user's side while cloud resources are used for the heavier computation.

However, this architecture requires repeated interaction between the edge and the cloud during both the prefill and decoding stages. When requests are processed sequentially, these interactions can create "bubbles"—periods in which computing resources remain idle while



Powered by
 HUAWEI

Enter | Register

Pay attention

Contest is running
ICPC 2026 Online Challenge 1 powered by Huawei
6 days
Register now »

Streams

CSES Dynamic Programming part 3
By Errichto
Before stream 04:45:29
View all →

Top rated

#	User	Rating
1	Benq	3857
2	jiangly	3812
3	maroonrk	3536
4	tourist	3530
5	Kevin114514	3510
6	turmax	3402
7	Um_nik	3389
8	Radewoosh	3367

Login or
Sign Up here!

Choose a cool name
(you'll be able to
change it exactly
once within the first
week)

CODEFORCES





Enter | Register

HOME TOP CATALOG **CONTESTS** **GYM** **PROBLEMSET** **GROUPS** RATING EDU API CALENDAR HELP

ICPC Challenge powered by Huawei

By ICPCNews, 3 weeks ago, 

Hello, Codeforces!

We are happy to invite you to an exciting online event: **ICPC Online Challenge powered by Huawei**.

This Challenge features the problem

Scheduling Algorithm for Collaborative Inference System With the rapid advancement of artificial intelligence, large language model (LLM) services are becoming increasingly integrated into people's daily lives and work. At the same time, the economic efficiency and data security of these services are receiving greater attention.

Edge-Cloud Collaborative Inference has emerged as a promising approach to addressing these challenges. In this architecture, LLM inference is divided between a user's on-premises edge environment and the cloud. Using a U-shaped split, the first and last layers of the model run on the edge, while the computationally intensive middle layers run in the cloud. This allows private inputs and outputs to remain on the user's side while cloud resources are used for the heavier computation.

However, this architecture requires repeated interaction between the edge and the cloud during both the prefill and decoding stages. When requests are processed sequentially, these interactions can create "bubbles"—periods in which computing resources remain idle while



Powered by  HUAWEI

→ Pay attention

Contest is running
ICPC 2026 Online Challenge 1 powered by Huawei
6 days
[Register now »](#)

→ Streams

CSES Dynamic Programming part 3
By **Errichto**
Before stream 04:45:29
[View all →](#)

→ Top rated

#	User	Rating
1	Benq	3857
2	jiangly	3812
3	maroonrk	3536
4	tourist	3530
5	Kevin114514	3510
6	turmax	3402
7	Um_nik	3389
8	Radevoosh	3367

Contests:
Core part of CP -
keep giving these
(loads of fun)

Problemset:
The GRIND (still
loads of fun)

Groups:
Prog Club contests!

CODEFORCES



JOIN THE GROUP!

<https://codeforces.com/group/du4g5GNE7E/>



CODEFORCES



CONTESTS

MEMBERS

Group Contests

Name	Start	Length
No contests		

Freshie Programmers 26-27

Private

Spectator

☆

-- About Group

The official codeforces group for IITM freshies managed by The Programming Club, IITM for the tenure 2026-27.

[Group website](#)

-- Member management

You are not group member yet, but can request group join.

Membership type: Participant

Join

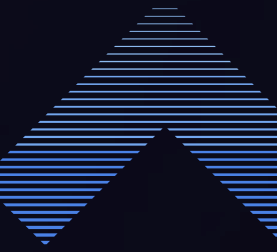
Join here!

CODEFORCES



**THERE'S A PROBLEMSET ON THE GROUP
FOR YOU TO TRY!**

**WE ENCOURAGE EVERYONE TO ATTEMPT
IT AND DON'T BE DISHEARTENED IF YOU
FAIL ON SOME PROBLEMS**



CODEFORCES



Starts on Aug 22, 7pm
Submissions Open till Aug 29, 7pm

Group Contests						
Name	Start	Length				
CPS-1 2026 Enter »	Aug/22/2026 22:00 ^{UTC+5.5}	7:00:00	Current standings Running 7 days Solved: 2 out of 8	Prepared by maxi_128 Registration completed x45	 Manager	

* Highlighted contests are not public
[Show deleted contests](#)

MAKE SURE TO JOIN THE GROUP
BEFORE REGISTERING AND THEN
CLICK ENTER



FEEDBACK FORM

[Click here to fill feedback form](#)





THANK YOU

COMING
SOON

CODE ARENA
2.0